

Lecture 7 — Memory Systems for AI Agents

Course notes · Building Autonomous Agents · Week 7 (demo document)

Learning objectives

- Explain why a context window is not a memory.
- Distinguish working, episodic, semantic and procedural memory and give an agent example of each.
- Compute a retrieval score by hand and predict which memories an agent will see.
- Describe consolidation and forgetting, and say why an agent needs both.
- Recognise the four classic memory failure modes in a transcript.

7.1 The context window is not memory

Every call to a language model starts from nothing. The model has knowledge, baked in during training, but it has no record of what you told it five minutes ago unless that record is in the prompt. The context window is therefore the agent's only state, and it has three properties that make it a poor long-term memory:

1. **It is small.** Even a window of a few hundred thousand tokens fills up in a working day of tool calls and observations.
2. **It is undifferentiated.** A hard constraint from the user and a verbose log line from a tool occupy the same kind of space, and nothing marks one as more important than the other.
3. **It is lost.** When the session ends, so does the window. The next session starts from nothing again.

Memory, in the sense we use in this course, is anything that survives outside the window and can be brought back in when it is needed. The design questions are what to write, when to read, and how to keep the store from becoming useless as it grows.

7.2 A taxonomy borrowed from psychology

Cognitive psychologists distinguish several memory systems, and the distinctions turn out to be exactly the ones an agent designer needs.

Memory system	Human example	Agent example	Typical implementation
Working	Holding a phone number while dialling	The current plan and last few observations	The prompt itself
Episodic	Remembering what you had for lunch on Tuesday	On 3 March the deploy failed because the token had expired	Event log with embeddings and time stamps
Semantic	Knowing that Paris is the capital of France	The user prefers concise answers and works in UTC+2	Profile document or key-value store
Procedural	Knowing how to ride a bicycle	How to file a bug report in this project	Tools, prompt templates, saved playbooks

Table 7.1. Memory systems and their agent counterparts.

A useful test: if a piece of information has a "when" attached to it, it is episodic. If it is true regardless of when you learned it, it is semantic. If it is a way of doing something rather than a thing you know, it is procedural. Working memory is whatever is currently in play.

Most agent frameworks ship with working memory (the prompt), some semantic memory (a system prompt written by a developer) and procedural memory (tools). Episodic memory is the piece that is usually missing, and it is the piece that makes an agent seem to remember you.

7.3 Retrieval: a worked example

Suppose the store contains the four memories below and the agent is about to answer the query "Which format should the weekly report use?". We compute a score for each memory as

$$\text{score} = \text{relevance} + 0.5 \times \text{recency} + 0.5 \times \text{importance}$$

with relevance the cosine similarity to the query, recency = $0.5^{\text{age in days}}$, and importance the stored score divided by 10.

Memory	Age	Relevance	Recency	Importance	Score
User asked for the report as a PDF, not a slide deck	6 days	0.82	0.02	0.8	1.23
Report generation tool succeeded	1 day	0.55	0.50	0.2	0.90
User mentioned the board meeting is on Friday	2 days	0.31	0.25	0.6	0.74
User said all documents must use the company template	20 days	0.60	0.00	0.9	1.05

Table 7.2. Retrieval scores for the query "Which format should the weekly report use?"

With $k = 2$ the agent sees the PDF request and the template rule, which is what we want: both are old, but both are relevant and important. The recent but trivial tool success is ranked below them. Notice what would happen with pure recency ordering: the tool success would be first and the template rule, at twenty days old, would never be retrieved at all. This is the argument for weighting importance.

The weights are not magic. Lower the importance weight and old constraints disappear; raise the recency weight and the agent becomes forgetful in a different way, obsessed with whatever happened last. In practice teams tune the weights on a handful of transcripts where they know what should have been retrieved.

7.4 Consolidation and forgetting

An episodic store grows every step. Two things must happen for it to stay useful.

Consolidation turns many episodes into a few facts. If the user has corrected the agent's date format three times, the three corrections should become one semantic entry: "dates are written day-month-year". This is a summarisation job that runs in the background, usually triggered when enough important events have accumulated since the last run. The original episodes are not deleted, but their weight is reduced so that the fact wins at retrieval time.

Forgetting is deliberate. The recency term already makes old episodes fade. Beyond that, most systems cap the store and evict the lowest-scoring episodes, and many delete episodes whose

content has been consolidated. The goal is not to save space; it is to keep retrieval clean. An agent that remembers everything retrieves the wrong thing.

Rule of thumb: anything the user says about how they want things done is a candidate for consolidation. Anything a tool says is a candidate for forgetting.

7.5 Failure modes to recognise

Failure	What you see in the transcript	Usual cause	Usual fix
Stale memory	Agent uses an address, name or decision that was later changed	The reversal was never written, or scored low	Write reversals as high-importance events; let consolidation overwrite
Retrieval miss	Agent asks for something it was already told	Query phrasing does not match the stored text	Embed time stamps and paraphrases; expand the query
Memory poisoning	Agent "remembers" an instruction that came from a web page or tool output	Provenance not recorded; untrusted text consolidated	Tag provenance; never consolidate untrusted sources
Over-retrieval	Prompt full of marginally related memories; agent ignores the current input	k too high for the size of the store	Lower k; let the agent ask for more memory on demand

Table 7.3. The four classic failure modes of agent memory.

Exercises

1. Classify each of the following as working, episodic, semantic or procedural memory: (a) the user's time zone; (b) the fact that the last API call returned a 429; (c) the steps for rotating a credential; (d) the plan for the current task.
2. Recompute Table 7.2 with the recency weight raised to 1.5. Which two memories are retrieved now, and what goes wrong?
3. A user tells the agent on Monday that they moved to Berlin. On Wednesday the agent books a meeting in the user's old time zone. Name the failure mode and propose two fixes, one at write time and one at consolidation time.
4. Explain in two sentences why a rolling summary of the transcript tends to lose hard constraints stated early in a session.
5. Give an example of a tool output that must never be consolidated into a semantic fact, and say why.

Key terms

- **Working memory** — the information an agent is actively using; in practice, the context window.
- **Episodic memory** — a store of specific, time-stamped events from the agent's own experience.
- **Semantic memory** — general facts and preferences, independent of when they were learned.
- **Procedural memory** — knowledge of how to do things: tools, templates, playbooks.
- **Importance score** — a model-assigned judgement, typically 1 to 10, of how much an event matters to future decisions.
- **Consolidation** — the background process that distils recurring episodes into semantic facts.

- **Retrieval score** — a weighted sum of relevance, recency and importance used to rank memories.
- **Memory poisoning** — the failure in which untrusted content is stored and later retrieved as if it were trusted.